WEBRUM-09: Migrating USQL to DQL

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 9 of 10 | **Created:** July 2026 | **Last Updated:** 10/23/2026

Overview

User Sessions Query Language (USQL) is Dynatrace's own Gen2 query language for RUM data. On Gen3 it is replaced by DQL — but unlike the competitor translations elsewhere in this corpus (SPL, NRQL, SumoQL), this one has **two possible targets**, and which one you need depends on the RUM generation your tenant serves.

That distinction is the whole notebook. Get it wrong and every query returns zero rows with no error to explain why.

	Classic RUM on Grail	New RUM
Session fields	`userActionCount`, `totalErrorCount`, `browserFamily`	`user_action_count`, `error.count`, `browser.name`
Event typing	`action.type == "Load"`	`characteristics.has_user_action == true`
What changes from USQL	**Grammar only** — field names carry over	Grammar **and** every field name

The useful surprise is on the left: classic RUM on Grail preserves the USQL field names almost verbatim, so that migration is a grammar exercise. Migrating to New RUM is a genuine field-by-field translation.

Table of Contents

1. Why This Is Two Migrations
2. Determining Which Model Your Tenant Serves
3. Grammar Mapping
4. Table Mapping
5. Field Mapping — Sessions

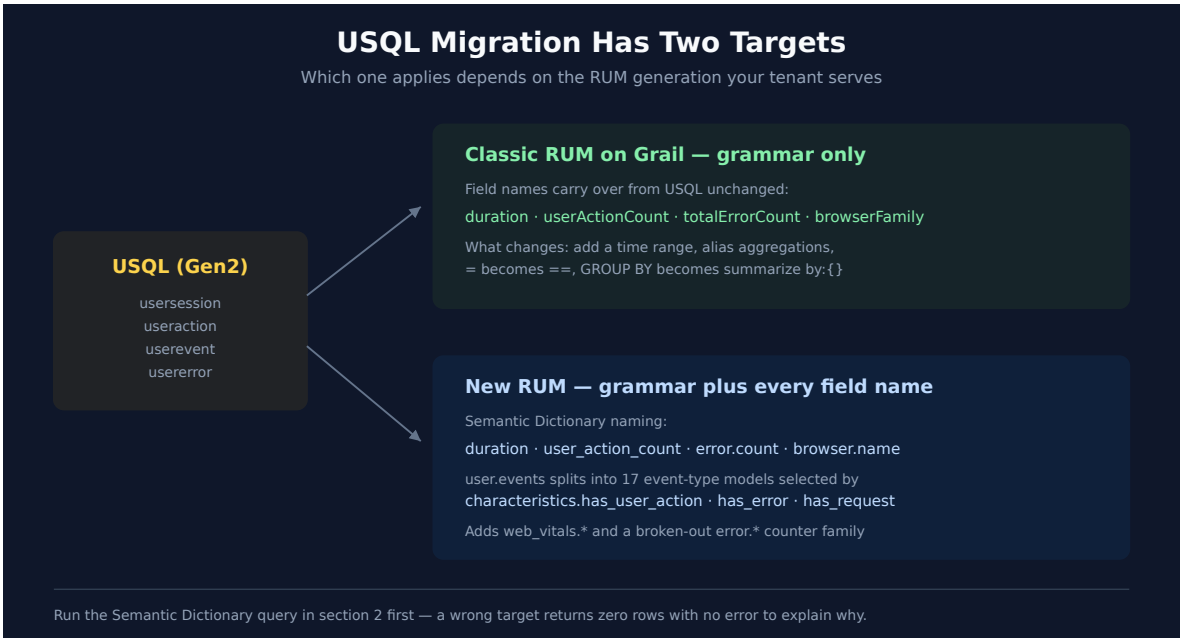
6. Field Mapping — User Actions
7. Worked Conversions
8. What Does Not Translate
9. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail
RUM data	At least one application reporting real-user sessions
Permissions	`storage:user.sessions:read` and `storage:user.events:read`
Prior reading	WEBRUM-01 (fundamentals), WEBRUM-04 (session analysis)

> **On the validation status of the queries in this notebook.** The field names on both sides are sourced from primary references — the USQL/classic column from the Dynatrace **User sessions API** documentation, the New RUM column from a live query against `dt.semantic_dictionary.models` on a Dynatrace tenant (07/23/2026). Both are authoritative for *what the fields are called*.
>
> The two discovery queries in section 2 were executed against a live tenant and returned the results described. The conversion examples in section 7 were **not** executed — the validation tenant holds zero records in `default_user_sessions` and `default_user_events`, so there was no real-user RUM data to run them against. They are schema-verified, not execution-verified. Run section 2 first, then confirm each conversion against your own data before relying on it.

1. Why This Is Two Migrations



USQL ran against a purpose-built session store. Gen3 moved RUM into Grail, and it did so in two stages:

- **Classic RUM on Grail** lifted the existing session model into Grail largely intact. In practice the USQL column names came along — `duration`, `userActionCount`, `totalErrorCount`, `userType` read as the same identifiers in both, which is how the classic queries in this series use them — so what changes is mainly the query grammar. That mapping is inferred rather than documented; confirm it with section 2's query before relying on it.
- **New RUM** is a redesign around the Dynatrace Semantic Dictionary. Fields are renamed to the platform's snake_case dotted conventions, and `user.events` is no longer one flat table of typed actions — it is a set of event-type models distinguished by `characteristics.has_*` flags.

The rest of the WEBRUM series is written against the New RUM vocabulary. On a classic-RUM-on-Grail tenant, use the second column of the tables below to translate it back.

> **Sources:** User session structure (DT docs), Custom queries, segmentation, and aggregation of session data (DT docs).

2. Determining Which Model Your Tenant Serves

Run this first. It decides which column of every table below applies to you.

The Semantic Dictionary is the authoritative description of the New RUM model. If it returns the `rum.user_session` model with snake_case dotted fields, your tenant knows about New RUM.

```
// Which RUM session model does this tenant describe?
// Snake_case dotted fields (browser.name, user_action_count) =&gt; New RUM.
fetch dt.semantic_dictionary.models
| filter data_object == "user.sessions"
| fields name, title, fields
```

The event side matters just as much. Under New RUM, `user.events` is described by a set of event-type models rather than one flat table — `rum_user_action`, `rum_page_summary`, `rum_navigation`, `rum_request`, `rum_exception`, `rum_crash`, and others. That is what replaces USQL's `useraction` / `userevent` / `usererror` split.

```
// Event-type models behind user.events under New RUM.
// Each is selected in a query via its characteristics.has_* flag.
fetch dt.semantic_dictionary.models
| filter data_object == "user.events"
| fields name, title
| sort name asc
```

Then confirm where the data actually is. A tenant can describe New RUM while its populated buckets still hold classic-shaped data, so check record counts rather than assuming.

```
// Which RUM buckets hold data, and can you read them?
// has_access == false means your permissions, not the data, are the problem.
fetch dt.system.buckets
| filter in(dt.system.table, {"user.sessions", "user.events"})
| fields name, dt.system.table, retention_days, records, has_access
| sort records desc
```

> **Reading the result.** Records in `default_user_sessions` and `default_user_events` are your real-user data. Records only in `default_synthetic_user_sessions` / `default_synthetic_user_events` mean the tenant is carrying synthetic traffic alone — conversions will parse and return nothing, which looks identical to a wrong field name. Confirm you have real-user records before concluding a query is broken.

3. Grammar Mapping

The grammar change is identical for both targets — only the field names in the third column differ.

USQL	DQL	Notes
`SELECT <fields>`	`fields <fields>`	Or omit — DQL returns all fields by default
`SELECT <agg>`	`summarize <alias> = <agg>`	**DQL requires an alias** if you sort on it afterwards

USQL	DQL	Notes
`FROM usersession`	`fetch user.sessions, from:-24h`	**The time range is mandatory in DQL**; USQL inherited it from the UI or API call
`WHERE <cond>`	`| filter <cond>`	`=` becomes `==`
`GROUP BY a, b`	`| summarize ..., by:{a, b}`	Curly braces, not bare list
`ORDER BY x DESC`	`| sort x desc`	Lowercase `desc`
`LIMIT 100`	`| limit 100`	USQL default 50, max 5000; DQL default 1000
`TOP(field, 10)`	`| summarize c = count(), by: {field} | sort c desc | limit 10`	No direct equivalent — decompose
`COUNT(*)`	`count()`	
`AVG` / `SUM` / `MIN` / `MAX`	`avg` / `sum` / `min` / `max`	Same names, lowercase
`MEDIAN(x)`	`percentile(x, 50)`	
`PERCENTILE(x, 90)`	`percentile(x, 90)`	
`DISTINCT`	`countDistinctExact()` / `collectDistinct()`	Depending on whether you want the count or the values
`LIKE '%x%'`	`contains(field, "x")`	
`STARTSWITH`	`startsWith(field, "x")`	
`IN (a, b)`	`in(field, {"a", "b"})`	Curly braces for the value set
`BETWEEN a AND b`	`field >= a and field <= b`	
`IS NULL` / `IS NOT NULL`	`isNull(field)` / `isNotNull(field)`	
`YEAR` / `MONTH` / `DAY` / `HOUR`	`getYear()` / `getMonth()` / `getDayOfMonth()` / `getHour()`	

Two rules cause most first-attempt failures:

1. **Every `fetch` needs a time range.** USQL took its window from the dashboard, the management-zone selector, or the API request. DQL does not — `fetch user.sessions` without `from:` is not the same query.
2. **Aggregations must be aliased before you sort on them.** `sort count() desc` is invalid; `summarize c = count()` then `sort c desc` is what USQL's implicit ordering becomes.

> **Sources:** Custom queries, segmentation, and aggregation of session data (DT docs) — USQL keyword, function, and operator lists.

4. Table Mapping

USQL had four tables. DQL has two, and the distinction USQL encoded in the table name moves into a filter.

USQL table	Classic RUM on Grail	New RUM
`usersession`	`fetch user.sessions`	`fetch user.sessions`
`useraction`	`fetch user.events` + `filter action.type == "Load"` (or `"Xhr"`, `"Click"`, …)	`fetch user.events` + `filter characteristics.has_user_action == true`
`userevent`	`fetch user.events`	`fetch user.events` + the relevant `characteristics.has_*` flag
`usererror`	`fetch user.events` + error-type filter	`fetch user.events` + `filter characteristics.has_error == true`

Under New RUM the event-type models each expose their own fields — `rum_exception` carries `error.name` / `error.type` / `error.is_fatal`, `rum_crash` adds `exception.stack_trace` and `exception.crash_signal_name`, `rum_request` carries `http.request.method` and `http.response.status_code`. Filter to the event type first, then reference its fields.

> **Sources:** live query of `dt.semantic_dictionary.models` (Dynatrace tenant, 07/23/2026) for the New RUM column; Custom queries, segmentation, and aggregation of session data (DT docs) for the USQL table list.

5. Field Mapping — Sessions

`usersession` → `user.sessions`. The classic column is **inferred, not documented**: it assumes the USQL name carries over unchanged, as the classic queries elsewhere in this series use them. Verify it against your own tenant per section 2 before relying on it.

USQL (`usersession`)	Classic RUM on Grail	New RUM
`duration`	`duration`	`duration`
`startTime` / `endTime`	`startTime` / `endTime`	`start_time` / `end_time`

USQL (`usersession`)	Classic RUM on Grail	New RUM
`endReason`	`endReason`	`end_reason`
`userActionCount`	`userActionCount`	`user_action_count`
`totalErrorCount`	`totalErrorCount`	`error.count`
`hasError`	`totalErrorCount > 0`	`error.count > 0`
`hasCrash`	`hasCrash`	`error.has_crash`
`userType`	`userType`	`dt.rum.user_type`
`userSessionId`	`sessionId`	`dt.rum.session.id`
`browserFamily`	`browserFamily`	`browser.name`
`browserMajorVersion`	`browserMajorVersion`	`browser.version`
`osFamily` / `osVersion`	`osFamily` / `osVersion`	`os.name` / `os.version`
`country`	`country`	`geo.country.iso_code`
`ip`	`ip`	`client.ip`
`isp`	`isp`	`client.isp`
`device`	`device`	`device.type` — **being removed; see note below**
`manufacturer`	`manufacturer`	`device.manufacturer`
`screenWidth` / `screenHeight`	`screenWidth` / `screenHeight`	`device.screen.width` / `device.screen.height`
`appVersion`	`appVersion`	`app.version`
`hasSessionReplay`	`hasSessionReplay`	`characteristics.has_replay`
`rootedOrJailbroken`	`rootedOrJailbroken`	`device.is_rooted`
`stringProperties.<key>`	`stringProperties.<key>`	`session_properties.<key>`
`bounce`	`userActionCount == 1`	`user_action_count == 1` — see note below

Fields with no direct New RUM equivalent in the session model: `city`, `continent`, `region` (only `geo.country.iso_code` is present), `userExperienceScore`, `matchingConversionGoals`, `newUser`, `numberOfRageClicks`. See section 8.

Bounce. Dynatrace defines a bounced session as *"A session with only one user action or navigation"*. The rest of this series counts it as `user_action_count == 1`, which keeps sessions with zero user

actions out of the bounce count; compare the result with the **Bounced Sessions** filter in Users & Sessions before treating the two as identical.

New RUM adds counters USQL had no equivalent for: `navigation_count`, `page_summary_count`, `request_count`, `user_interaction_count`, `view_summary_count`, and a broken-out error family (`error.exception_count`, `error.http_4xx_count`, `error.http_5xx_count`, `error.csp_violation_count`, `error.anr_count`).

> **Sources:** User session structure (DT docs) for the USQL column; live query of `dt.semantic_dictionary.models` (model `rum.user_session`, Dynatrace tenant, 07/23/2026) for the New RUM column; User sessions in web frontends (DT docs) — *"A session with only one user action or navigation is tagged as bounced."*
>
> **`device.type` is being removed from the user-session model (Semantic Dictionary 1.348, released 08/25/2026).** A migrated USQL `device` query that lands on `device.type` stops resolving once 1.348 reaches your tenant.
>
> **No replacement is documented.** The same version adds a `client.*` family (`client.type`, `client.version`, `client.application.id`, and others) which *looks* like the successor, but the changelog states no mapping and this entry will not assert one. Verified against a live tenant on 09/09/2026: that tenant is still pre-1.348 — `device.type` is present on the `rum.user_session` model (`user.sessions`) and no `client.type` field exists yet — so the row above is accurate today and cannot responsibly be re-pointed until 1.348 lands.
>
> The removal is scoped to the **user-session** model. `device.type` also sits on three `dt.system.events` RUM billing models (`real_user_monitoring_usage_event` and siblings) that the changelog does not touch, so consumption queries splitting by it are unaffected — see FINOPS-01.

6. Field Mapping — User Actions

`useraction` → `user.events`. This is where the two targets diverge most.

USQL (`useraction`)	Classic RUM on Grail	New RUM
`name`	`action.name`	`interaction.name` / `ui_element.name`
`type`	`action.type`	`characteristics.has_*` flags (`has_page_summary`, `has_navigation`, `has_request`, …) — not `characteristics.classifier`
`duration`	`duration`	`duration`
`application`	`application`	`dt.rum.application.entities`

USQL (`useraction`)	Classic RUM on Grail	New RUM
`largestContentfulPaint`	`largestContentfulPaint`	`web_vitals.largest_contentful_paint`
`firstInputDelay`	`firstInputDelay`	`web_vitals.first_input_delay`
`cumulativeLayoutShift`	`cumulativeLayoutShift`	`web_vitals.cumulative_layout_shift`
(no equivalent)	*(no equivalent)*	`web_vitals.interaction_to_next_paint`
(no equivalent)	*(no equivalent)*	`web_vitals.time_to_first_byte` (on `rum_request`)
`javascriptErrorCount`	`javascriptErrorCount`	count `rum_exception` events
`requestErrorCount`	`requestErrorCount`	`characteristics.has_failed_request`
`hasCrash`	`hasCrash`	`characteristics.has_crash`

Request-shaped actions (USQL `type == 'Xhr'`) map onto the `rum_request` model, which exposes `http.request.method` , `http.response.status_code` , and `http.response.reason_phrase` — none of which existed as USQL user-action fields.

> **Select `user.events` with the `characteristics.has_*` flags, not `characteristics.classifier` .** The User events semantic-dictionary page describes `characteristics.classifier` as the single characteristic picked by priority when an event has several, and says it is *"Used for internal optimization when storing the data and not intended for query usage."* **Semantic Dictionary 1.349** (published 09/04/2026) removes it from the user-event models, with a **staged tenant rollout** — a tenant that has not reached 1.349 still returns the field, so a classifier filter keeps working until the change arrives and then silently matches nothing. The stable flags (`has_page_summary` , `has_navigation` , `has_error` , `has_request` , …) are the documented selectors and work on both sides of the change. **Semantic Dictionary 1.350** (published 09/21/2026) re-lists `characteristics.classifier` under the Basic user-event model, still `experimental` , and the User events page still calls it *"not intended for query usage"* — so the flags above remain the selectors.
>
> The flags can **overlap** where the classifier picked one winner. A navigation that also carried an error was classified `error` , so `classifier == "navigation"` silently dropped it; `characteristics.has_navigation == true` keeps it. On the validation tenant (24 h, 09/28/2026) that is **7,387** navigations against **6,235** — the 1,152 recovered page loads have full navigation timing. For page summaries the two filters select the same 7,268 events.
>
> **Sources:** User session structure (DT docs) for the documented user-action fields; live query of `dt.semantic_dictionary.models` (models `rum_user_action` , `rum_request` , `rum_exception` , `rum_crash` , Dynatrace tenant, 07/23/2026) for the New RUM column; User events — semantic dictionary (DT docs) — *"Used for internal optimization when storing the data and not intended for query usage."*; Semantic Dictionary changelog 1.349 (DT docs), Semantic Dictionary changelog 1.350 (DT docs). **Dictionary:** models

> `rum_page_summary` / `rum_navigation` list `characteristics.has_page_summary` / `characteristics.has_navigation`, read 09/28/2026.

7. Worked Conversions

Each cell shows the original USQL as a comment, then the DQL for both targets. Uncomment the one matching your tenant.

> These were not executed against real-user data — see the note in Prerequisites. Verify against your own tenant.

```
// USQL: SELECT browserFamily, COUNT(*) FROM usersession GROUP BY browserFamily

// --- Classic RUM on Grail ---
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| summarize session_count = count(), by:{browserFamily}
| sort session_count desc
| limit 10

// --- New RUM ---
// fetch user.sessions, from:-24h
// | filter dt.rum.user_type == "real_user"   // documented values: real_user / robot /
synthetic
// | summarize session_count = count(), by:{browser.name}
// | sort session_count desc
// | limit 10
```

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM). Classic camelCase
RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt; user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; frontend.name (stable; an array on user.sessions, so expand it
before grouping)
//                  primary_tags.application exists only if your OpenPipeline adds it
(corrected 10/14/2026)
//   country/city/continent -&gt; geo.country.name / geo.city.name / geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.value (page summaries; web_vitals.time_to_first_byte where
populated) —
//                  NOT ttfb.waiting_duration, which is the pre-request wait/redirect
phase
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT query
empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter matches
nothing —
//     the documented values are "real_user" / "robot" / "synthetic" (lowercase);
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// USQL: SELECT AVG(duration) FROM usersession WHERE browserFamily = 'Chrome'
// Note the two changes: = becomes ==, and the aggregation gets an alias.

// --- Classic RUM on Grail ---
fetch user.sessions, from:-24h
| filter browser.name == "Chrome"
| summarize avg_duration = avg(duration)

// --- New RUM ---
// fetch user.sessions, from:-24h
// | filter browser.name == "Chrome"
// | summarize avg_duration = avg(duration)
```

```
// USQL: SELECT country, COUNT(*) FROM usersession WHERE hasError IS TRUE GROUP BY
country
// hasError has no DQL equivalent — it becomes a count comparison.

// --- Classic RUM on Grail ---
fetch user.sessions, from:-24h
| filter userType == "REAL_USER"
| summarize {total = count(),
    with_errors = countIf(totalErrorCount &gt; 0)},
    by:{country}
| fieldsAdd error_pct = round(toDouble(with_errors) / toDouble(total) * 100.0, decimals:
1)
| sort error_pct desc
| limit 20

// --- New RUM ---
// fetch user.sessions, from:-24h
// | summarize total = count(),
//     with_errors = countIf(error.count &gt; 0),
//     by:{geo.country.iso_code}
// | fieldsAdd error_pct = round(toDouble(with_errors) / toDouble(total) * 100.0,
decimals: 1)
// | sort error_pct desc
// | limit 20
```

```
// USQL: SELECT TOP(name, 10), COUNT(*) FROM useraction GROUP BY name
// TOP() decomposes into summarize + sort + limit.

// --- Classic RUM on Grail ---
fetch user.events, from:-24h
| filter action.type == "Load"
| summarize action_count = count(), by:{action.name}
| sort action_count desc
| limit 10

// --- New RUM ---
// fetch user.events, from:-24h
// | filter characteristics.has_user_action == true
// | summarize action_count = count(), by:{interaction.name}
// | sort action_count desc
// | limit 10
```

```
// USQL: SELECT AVG(largestContentfulPaint) FROM useraction WHERE type = 'Load' GROUP BY
application
// Web Vitals move into the web_vitals.* namespace under New RUM.

// --- Classic RUM on Grail ---
fetch user.events, from:-24h
| filter action.type == "Load"
| filter isNotNull(largestContentfulPaint)
| summarize {avg_lcp = avg(largestContentfulPaint),
    p90_lcp = percentile(largestContentfulPaint, 90)},
    by:{application}
| sort p90_lcp desc

// --- New RUM ---
// fetch user.events, from:-24h
// | filter characteristics.has_user_action == true
// | filter isNotNull(web_vitals.largest_contentful_paint)
// | summarize avg_lcp = avg(web_vitals.largest_contentful_paint),
//     p90_lcp = percentile(web_vitals.largest_contentful_paint, 90),
//     by:{dt.rum.application.entities}
// | sort p90_lcp desc
```

8. What Does Not Translate

USQL feature	Status	What to do instead
`FUNNEL`	No DQL equivalent	Rebuild as sequential `countIf()` stages, or model the funnel as business events — see the BIZEV series
`matchingConversionGoals`	Not in the New RUM session model	Conversion goals are a classic-application construct; express the goal as a filter on the action or as a business event
`userExperienceScore`	Not in the New RUM session model	Apdex-style classification has to be derived from duration thresholds
`city` / `continent` / `region`	Only `geo.country.iso_code` in New RUM	Country-level granularity only, unless enriched at ingest
`newUser`	Not in the New RUM session model	Derive from user-id first-seen, or track as a session property
`numberOfRageClicks` / `numberOfRageTaps`	Not a session-level counter in New RUM	Count the corresponding event-type records

USQL feature	Status	What to do instead
`KEYS` / `CONDITION`	No DQL equivalent	`KEYS` was USQL property discovery; use `fieldsSummary` or the Semantic Dictionary

Two behavioural differences worth knowing before you trust a result:

- **Session windowing.** `fetch user.sessions, from:-24h` returns sessions by their timestamp within the window. Long sessions can begin outside a narrow window, so a correlation query over a short range can under-count. Widen the lookback and filter afterwards when session boundaries matter.
- **Closed sessions only.** USQL documented that only closed sessions are queryable. Expect a comparable lag before an in-flight session is complete in Grail, and do not treat the most recent minutes as final.

If `fetch user.sessions` returns nothing at all, work through section 2 before assuming a field name is wrong — an unpopulated bucket, a missing `storage:user.sessions:read` scope, and a wrong field name all produce an empty result set, and only one of them is a query problem.

> **Sources:** Custom queries, segmentation, and aggregation of session data (DT docs) — `FUNNEL` limits, closed-session constraint, `KEYS` / `CONDITION`; live query of `dt.semantic_dictionary.models` for the absent-field determinations.

9. Summary and Next Steps

The three things to carry away:

1. **Establish your target before translating anything.** Section 2 takes a minute and determines which column of every table applies. Skipping it produces queries that parse cleanly and return nothing.
2. **On classic RUM, this is a grammar migration.** The USQL field names carry over. Fix the time range, alias the aggregations, swap `=` for `==`, and most queries work.
3. **On New RUM, budget for a rewrite.** Field names change, and the `useraction` / `userevent` / `usererror` split becomes `characteristics.has_*` filters over event-type models with their own field sets.

Where to go next:

If you need…	Read
Session analysis patterns on Gen3	WEBRUM-04 (session analysis)
Error and crash analysis	WEBRUM-05 (error analysis)

If you need…	Read
Core Web Vitals in depth	WEBRUM-03 (core web vitals)
Funnels and conversion analytics	The BIZEV series
DQL syntax generally	ORGNZ-99 (best practice summary and DQL reference)
The wider classic-to-Gen3 migration this sits inside	The -START-HERE- playbook, Doorway 4

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.